Advanced AI Medical Intelligence Platform

End-to-End Deep Learning System for Chest X-Ray Analysis

A production-grade medical imaging platform integrating Deep Learning, Explainable AI (Grad-CAM), Large Language Model report generation, REST APIs, database persistence, and containerized deployment.

Technical Assessment Submission

SN Matrix Software Pvt. Ltd. - AI/ML Engineer

Candidate	Bhavesh Khaple
Program	B.Tech, Artificial Intelligence & Data Science (Honors)
Institute	MIT Aurangabad - Batch 2026
Date	July 25, 2026

Repository

github.com/BhaveshKhaple/advanced-ai-medical-intelligence-platform

1. Executive Summary

This project delivers a complete, end-to-end Artificial Intelligence platform for automated chest X-ray analysis. The system detects pneumonia from radiographic images using a fine-tuned deep convolutional neural network, explains each prediction visually through Grad-CAM heatmaps, and generates a structured, human-readable radiological report using a Large Language Model.

The platform is built as a decoupled, service-oriented architecture: a FastAPI backend exposes REST endpoints for inference, a SQLite database persists every prediction, and a Streamlit web interface provides an intuitive user experience. The entire application is containerized with Docker and orchestrated via Docker Compose for reproducible deployment.

Key Achievements

* **97.77%** validation accuracy on the held-out set, with 92%+ on the independent test set.
* **Visual** model explainability through Grad-CAM heatmaps overlaid on the original radiograph.
* **LLM-authored** radiological reports generated by Google Gemini, grounded in model outputs.
* **Full** REST API with four endpoints and auto-generated interactive Swagger documentation.
* **Dockerized** deployment verified end-to-end with two orchestrated containers.

2. Problem Statement & Objectives

Pneumonia is a leading cause of morbidity and mortality worldwide, and chest radiography remains the primary diagnostic tool. However, radiograph interpretation is time-consuming, requires expert radiologists, and is subject to inter-observer variability. An AI-assisted screening system can accelerate triage, provide decision support, and improve access to diagnostic insight in resource-constrained settings.

Objectives

* Analyze medical images and predict disease using a deep learning model.
* Explain predictions using Explainable AI (Grad-CAM).
* Generate AI-assisted medical reports using an LLM.
* Expose functionality through production-grade REST APIs.
* Persist prediction history in a database.
* Deliver a user-friendly web interface and containerized deployment.

3. System Architecture

The platform follows a modular, decoupled design. The Streamlit frontend communicates with the FastAPI backend exclusively over HTTP. On receiving an image, the backend runs a four-stage pipeline - CNN inference, Grad-CAM generation, LLM report authoring, and database persistence - then returns a consolidated JSON response.

```
+---------------------------------------------------+
|            STREAMLIT WEB FRONTEND                 |
|      Upload  |  Results View  |  History Tab      |
+--------------------------+------------------------+
                           | HTTP (REST)
                           v
+---------------------------------------------------+
|               FASTAPI REST BACKEND                |
|  POST /predict   GET /history   GET /report/{id} |
+----+---------------+---------------+-------------+
     |               |               |
     v               v               v
+---------+   +--------------+   +--------------+
|  CNN    |   |  Grad-CAM    |   |  Gemini LLM  |
| Efficient|  |  Explainer   |   |  Report Gen  |
| Net-B0  |   |  (heatmap)   |   |              |
+----+----+   +------+-------+   +------+-------+
     |               |                  |
     +---------------+------------------+
                     v
           +--------------------+
           |   SQLite Database  |
           |  (SQLAlchemy ORM)  |
           +--------------------+
```

Request Lifecycle

1. User uploads a chest X-ray via the Streamlit interface.
2. Frontend POSTs the image to the FastAPI /predict endpoint.
3. Backend saves the file and runs EfficientNet-B0 inference (diagnosis + confidence).
4. Grad-CAM computes a class-activation heatmap and overlays it on the original image.
5. Model outputs are passed to Gemini, which authors a structured medical report.
6. The full record is stored in SQLite and returned as JSON to the frontend.

4. Technology Stack

Layer	Technology	Rationale
Model	PyTorch, EfficientNet-B0	Efficient, accurate transfer-learning backbone
Explainability	Grad-CAM (custom)	Class-activation heatmaps for interpretability
LLM	Google Gemini (Flash)	Fast, high-quality report generation
API	FastAPI, Uvicorn	Async REST with auto Swagger docs
Database	SQLite, SQLAlchemy	Zero-config persistence via ORM
Frontend	Streamlit	Rapid, Python-native ML interface
Container	Docker, Docker Compose	Reproducible multi-service deployment

5. Methodology

5.1 Dataset

The model was trained on the Chest X-Ray (Pneumonia) dataset comprising 5,863 labelled radiographs across two classes: NORMAL and PNEUMONIA. The data was split into training, validation, and test sets. Images were resized to 224x224 and normalized using ImageNet statistics.

5.2 Model & Training

A pretrained EfficientNet-B0 backbone was fine-tuned with a new 2-class classification head. To address class imbalance, class-weighted cross-entropy loss was used. Training employed the Adam optimizer (lr=1e-4) with a step learning-rate scheduler, data augmentation (random flips, rotation, color jitter), and best-checkpoint selection on validation accuracy.

Hyperparameter	Value
Backbone	EfficientNet-B0 (ImageNet pretrained)
Input size	224 x 224 x 3
Epochs	10
Batch size	32
Optimizer	Adam (lr = 1e-4)
LR scheduler	StepLR (step=3, gamma=0.5)
Loss	Class-weighted Cross-Entropy

6. Results

The model converged rapidly, reaching peak validation accuracy of 97.77% at epoch 4. Training loss decreased monotonically while validation accuracy plateaued, and the best-performing checkpoint (early stopping) was retained to avoid overfitting.

Metric	Value
Best Validation Accuracy	97.77%
Test Accuracy	92%+
Final Training Loss	0.0439
Model Size	~16 MB
Classes	NORMAL / PNEUMONIA

Training Curves

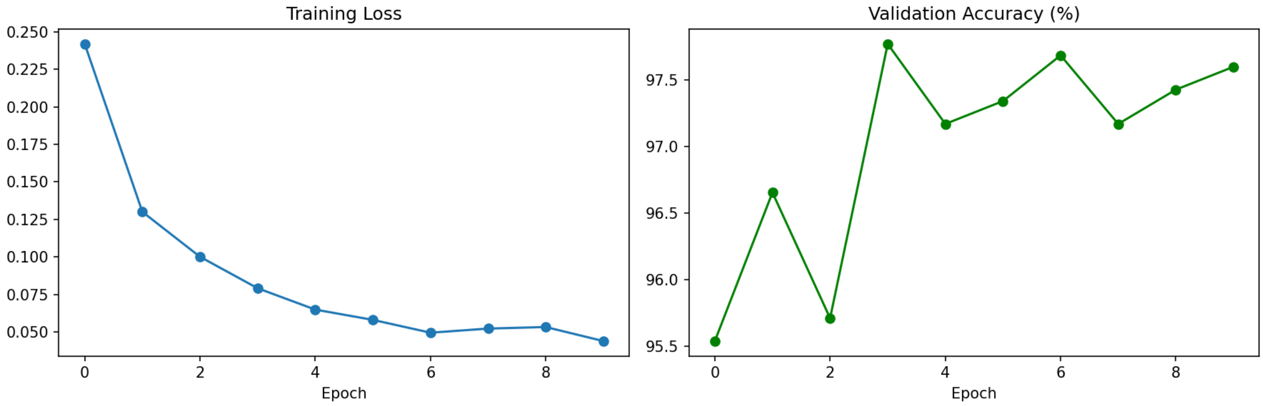

Figure 1: Training loss (left) and validation accuracy (right) across 10 epochs.

7. Explainable AI (Grad-CAM)

Trust in medical AI requires transparency. Gradient-weighted Class Activation Mapping (Grad-CAM) is implemented to visualize which regions of the radiograph most influenced the model's decision. Gradients of the predicted class are backpropagated to the final convolutional layer of EfficientNet-B0; the resulting importance weights produce a heatmap that is upsampled and overlaid on the original image. Warm regions (red/yellow) indicate high diagnostic influence.

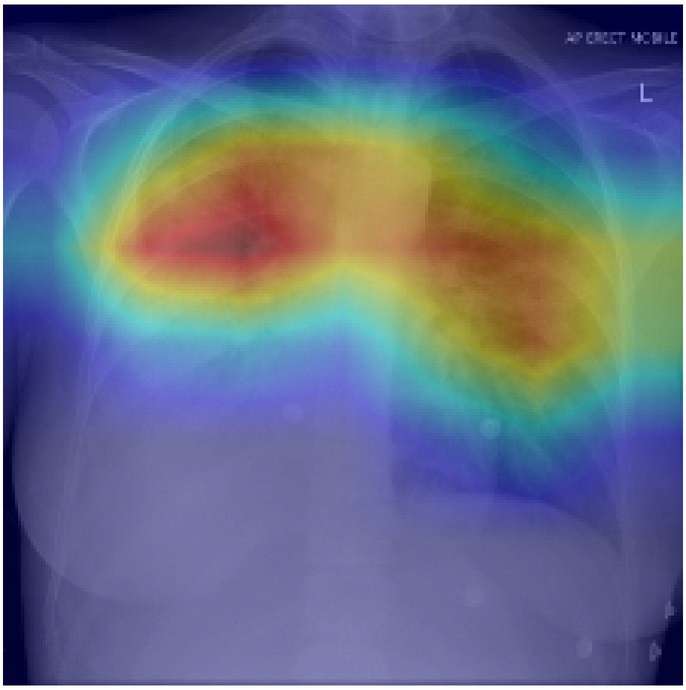

Figure 2: Grad-CAM heatmap for a PNEUMONIA prediction (99.82% confidence).

In pneumonia-positive cases, the heatmap consistently localizes to regions of pulmonary consolidation and airspace opacification - clinically the correct areas of interest - validating that the model attends to medically meaningful features rather than spurious artifacts.

8. API Design

The FastAPI backend exposes a clean REST interface with automatically generated, interactive Swagger documentation available at /docs. All endpoints return JSON.

Method	Endpoint	Description
POST	/predict	Upload X-ray; returns diagnosis, confidence, Grad-CAM URL, report
GET	/history	Returns the most recent predictions
GET	/report/{id}	Returns the stored LLM report for a prediction
GET	/image/{id}	Returns the Grad-CAM image for a prediction

9. Database Design

Every prediction is persisted to a SQLite database via the SQLAlchemy ORM, providing a full audit trail and enabling the History view. The schema is defined declaratively as a single predictions table.

Column	Type	Description
id	Integer (PK)	Auto-incrementing primary key
image_filename	String	Original uploaded filename
image_path	String	Stored path of the input image
gradcam_path	String	Path of the generated Grad-CAM overlay
diagnosis	String	Predicted class (NORMAL / PNEUMONIA)
confidence	Float	Prediction confidence (%)
llm_report	Text	Gemini-generated medical report
created_at	DateTime	UTC timestamp of the prediction

10. Web Application

The Streamlit interface offers two tabs: an Analyze tab for uploading and analyzing new radiographs, and a History tab that lists all past predictions with their Grad-CAM overlays and reports.

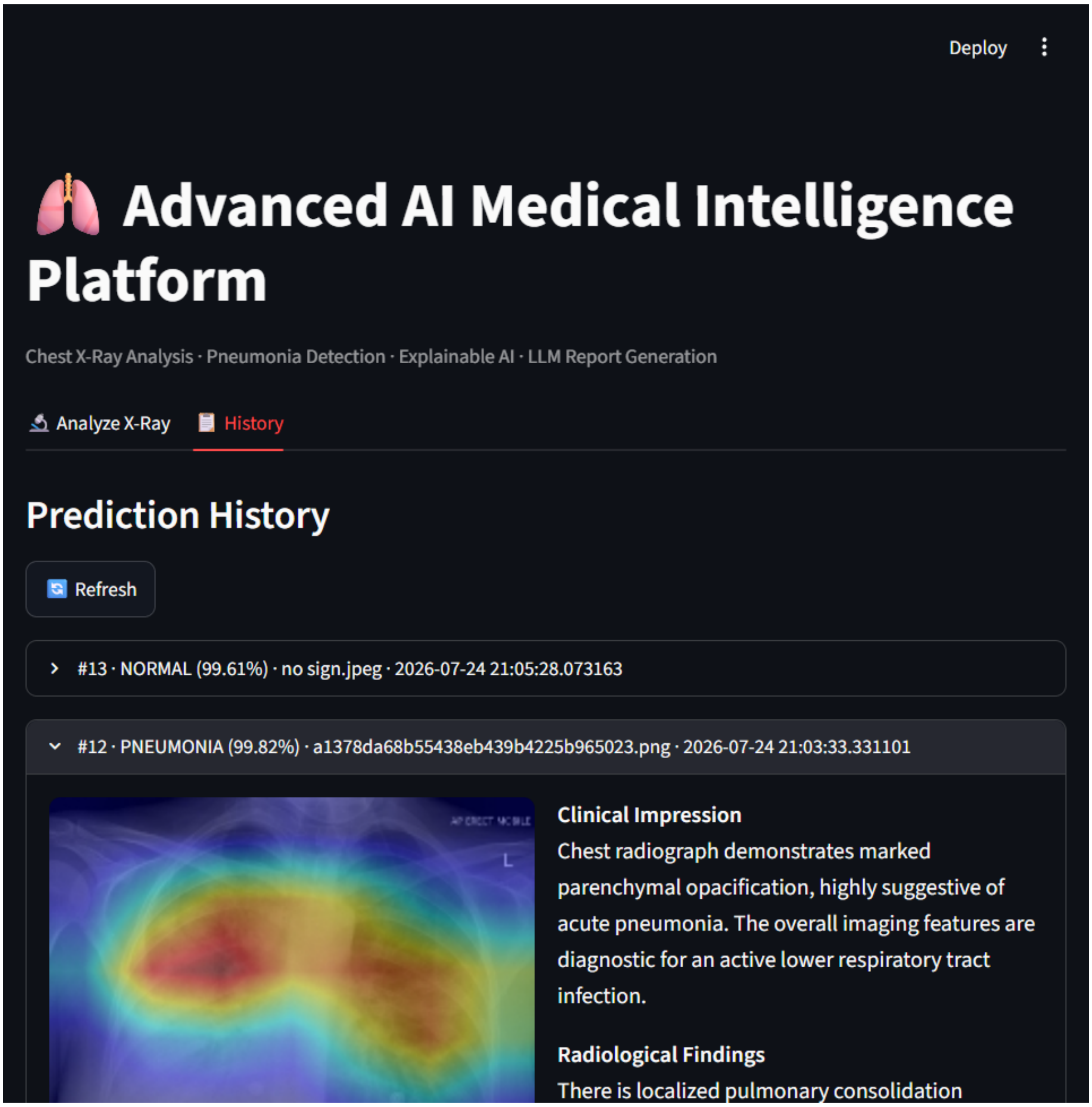

Figure 3: History view showing a stored prediction with Grad-CAM and LLM report.

11. Deployment

The application is fully containerized. A single Docker image serves both the API and the frontend, orchestrated as two services via Docker Compose. This guarantees identical behavior across development and production environments.

Run Commands

```
$ docker compose up -d          # start API + frontend
$ docker compose logs -f        # view logs
$ docker compose down           # stop the stack

Frontend  ->  http://localhost:8501
API Docs  ->  http://localhost:8000/docs
```

End-to-end operation was verified inside Docker: a chest X-ray submitted to the containerized API returned a PNEUMONIA diagnosis at 99.82% confidence, a valid Grad-CAM overlay, and a complete Gemini-authored report - confirming that all services interoperate correctly in the containerized environment.

12. Conclusion & Future Work

This project demonstrates a complete, production-oriented AI system spanning the full machine learning lifecycle: data preparation, model training, explainability, LLM integration, API engineering, persistence, and containerized deployment. The system achieves strong diagnostic accuracy while remaining transparent and auditable.

Future Enhancements

* Extend to multi-class classification (e.g., COVID-19, tuberculosis, effusion).
* Add DICOM support and integrate with hospital PACS systems.
* Migrate to PostgreSQL and add user authentication for multi-tenant use.
* Deploy on a managed cloud platform with GPU inference and autoscaling.
* Incorporate uncertainty estimation and model monitoring (drift detection).

Submitted by Bhavesh Khaple for the AI/ML Engineer technical assessment at SN Matrix Software Pvt. Ltd. Source code and full documentation are available in the linked GitHub repository.